

摘要：

Anthropic率先在模型跨过能力阈值后释放产品形态。关键在于设置连接研究与产品的特殊岗位，通过Evals将用户反馈转化为训练标准，并依托原型文化让Claude Code从个人工具演变为正式产品。模型与产品在co-design loop中共同演进，让其跑出了复利。

据彭博社最新报道，截至今年7月底，**Anthropic年化营收运行率已经突破650亿美元**。与2025年底约90亿美元的水平相比，这一数字在7个月内增长超7倍。

刚刚结束的二季度，Anthropic初步收入也已超过115亿美元，同比增幅达到14倍，并首次实现营业利润转正。

在前沿模型公司普遍依赖巨额融资、持续消耗算力的阶段，这是一条罕见的增长曲线。

## 7个月，Anthropic 年化营收增长超过7倍

年化营收运行率（ARR）

单位：亿美元  
纵轴为对数坐标



资料来源：彭博社（2026 年 7 月）、The Economist / Gurgavin（历史数据）。注：ARR 为按当前收入速度折算的年化运行率，不等于已确认的全年收入。

深流研究所  
公众号：深流研究所

推动增长的当然有Claude模型本身，但**模型能力并不会自动转化为收入**。

过去几年，Anthropic一面推进前沿模型能力，一面推出Claude Code、MCP、Cowork等带有新品类意味的产品。

这家公司常常能在模型跨过能力阈值后，迅速找到释放能力的产品形态，并实现商业化。这很难归因于几次偶然的产品判断。

毕竟，押注 Coding 的公司并不少，Anthropic 却率先把前沿模型变成了可用、可验证的产品，并跑出了复利。**它是怎么做到的？**

## 1、在研究和产品之间，Anthropic设置了一种特殊岗位

要理解这些问题，可以先从Anthropic内部一种特殊的岗位说起。

Dianne Penn是**Anthropic第一位技术产品经理**。她在2023年加入Anthropic，公开使用过的职务名称包括Head of Product for AI Research and Labs、Head of Product for Research and Labs，以及 Head of Research Product Management。

这些说法虽有差异，却都指向一个特殊的交叉点：一端进入AI Research，跟进模型研究团队的工作；另一端连接产品实验室Labs，用原型探索新能力可能长出什么产品。

在传统软件公司，技术地基相对稳定，产品经理可以围绕明确需求规划功能。但在前沿模型公司，**这块地基每隔几个月就可能改变**。

今天做不到的产品，下一代模型可能突然做到了；当下为弥补模型缺陷搭建的复杂流程，半年后也可能成为限制新模型的手脚架。

Dianne和团队的核心工作，正是研究和管理这种不确定性：

实验室里刚出现的能力，哪些已经接近可用？用户说“Claude不好用”，缺的是模型、工具、界面，还是工程基础设施？当用户拿Claude完成团队未曾设想的任务时，那是偶然技巧，还是一种新的产品可能？

Dianne需要把这些判断带回研究、产品和工程团队。

## 2、如何把用户的一句「不好用」，变成模型的训练目标？

2023年7月，Claude 2上线后的一次bug排查，初步展示了这套工作方式如何运转。

当时，用户频繁抱怨Claude 2“不太会遵循指令”。这个反馈看似严重，却无法直接指导模型研究。用户不会区分问题来自预训练、后训练、还是系统提示、或者工具调用。他们只能描述任务没有完成。

Dianne和团队开始阅读用户授权分享的失败轨迹，追问任务究竟在哪一步停下。

沿着案例向下查，他们发现，在早期这类投诉中，**大约80%都指向一个具体得近乎琐碎的问题：Claude无法稳定输出正确的JSON**。

少一个括号，API就会拒绝请求。字段不符合schema，工具调用便无法解析。

对聊天机器人而言，这些只是格式错误；对需要连续调用工具的Agent而言，它意味着工作流会在第一道接口处断掉。

团队从真实失败中收集了约30—40个案例，逐一写清输入、理想输出和通过条件，把它们变成一组可以重复运行的eval。此后，Claude每一代模型都要重新接受测试。

按照Dianne在公开采访中的讲述，这组评测如今已达到接近99.9%—100%的通过率，JSON格式遵循也不再是主要投诉。

Anthropic没有把这些反馈停留在投诉归类层面。



团队把模糊的“不好用”还原为具体任务轨迹，定位可复现的失败，再将失败案例写成可重复运行的eval。这些评测由研究与产品团队共同维护，逐渐成为模型能力的一部分规格。

Dianne常说：“**Evals are the new PRDs.**”

传统PRD可以描述功能，却很难精确定义一个概率系统应该如何行动；eval则把产品目标变成可测试的输入、理想输出和通过条件。

研究团队用它比较训练方法，产品团队用它验证问题是否解决，模型发布团队则用它防止能力退步。产品愿景由此进入研究和训练管线，几支团队也有了讨论问题的共同尺度。

### 3、从用户行为到训练方向，Coding 如何成为战略重点？

2023年末，Dianne和团队又注意到一种变化。

过去，人们使用代码模型，主要是补一行代码、写一个函数，或者解释一段程序。

后来，越来越多用户开始要求Claude一次生成更长、更完整的代码，甚至把一整段编码任务交给模型。

**没有人直接要求Anthropic“做一个agentic coding产品”，但用户已经改变了自己的行为。**

对此，Dianne和团队需要判断：这种变化究竟是噪声，还是真实存在的场景需求。

Dianne回忆，预训练、推理、研究和产品团队会一起讨论，这种能力改善是否稳定，哪些失败需要进入评测，并成为下一代模型要解决的问题。

在Dianne 看来，编程尤其适合形成这样的反馈闭环。代码能否运行、测试能否通过、改动是否破坏现有系统，通常都有明确结果。与许多开放式知识任务相比，**coding更容易同时成为产品场景、训练方向和评测环境。**

发现这类用户行为，Anthropic 把长代码生成提升为 Claude 3 系列及后续模型的重点能力方向。

Anthropic没有披露具体训练配方，但可以确认的是，coding获得了更高优先级。团队建立了更贴近长任务的评测，并在模型测试和发布判断中更加重视它。

2024年3月，Claude 3 Opus发布。Coding开始成为Claude区别于其他模型的一项用户认知，并吸引了一批早期开发者和Claude拥护者。

三个月后，Claude 3.5 Sonnet发布，coding能力进一步增强。Anthropic官方说明，Claude 3.5 Sonnet将内部agentic coding评测的成绩**从38%推高至64%**；获得工具后，模型已经可以编写、编辑、执行和排查代码。

模型的角色开始从代码生成器变成软件工程协作者。但用户仍要手动搬运代码库上下文、复制结果、执行命令，再把报错贴回聊天框。

Dianne把这种状态称为“**product overhang**”——模型已经拥有潜在能力，市场上却没有合适的产品把它释放出来。



## 4、先做出原型，再让真实使用决定产品去留

Claude Code的出现，填补了这段空白。

2024年下半年，Boris Cherny加入Anthropic Labs团队。为了熟悉公司的公开API，他给自己写了一个极小的终端聊天程序：先给模型接上bash，后来又增加文件编辑能力，用Sonnet 3.5不断测试。**这就是Claude Code最初的原型。**

两天后，Boris把原型发给同事试用。第二天，他发现坐在对面的工程师已经用它写起了代码。Boris没有推动正式的内部部署，只发过一篇介绍帖，工具却在工程师之间口口相传。

在一次内部产品评审会上，看到日活曲线近乎垂直增长，Anthropic创始人Dario发出质问，他们是不是在强迫员工使用。

最初约三个月，Claude Code基本由Boris一人推进。它的许多重要功能，并不是团队在会议室里提前规划出来的，**而是用户先用自己的办法补足产品缺口。**

有人会写一个Markdown文件，记录项目结构、编码规范、测试命令和注意事项，要求Claude每次先阅读；另一些用户则反复提醒Claude：“先不要写代码，先讨论思路，给我一个计划。”

当这些行为反复出现，团队便把它们从提示技巧收拢为正式产品能力：**前者成为CLAUDE.md，后者发展为Plan mode。**

Claude Code在内部迅速普及后，Anthropic还面临一个选择：是否把它保留为内部生产力优势？

Boris主张将它开放出去，因为实验室里的合成eval虽然可以控制变量，却无法还原模型进入真实代码库后会发生什么。

2025年2月，Claude Code以研究预览版推出。Anthropic官方介绍，它既是一款产品，**也是一套收集真实工作流反馈的研究工具。**

## 5、只用10天，Anthropic为什么就能做出Cowork？

Claude Code发布后，Boris不断看到用户拿它做与编程无关的事。

有人用它规划旅行、整理邮件和制作幻灯片，也有人让它从损坏的硬盘中找回照片，或接入摄像头和家用设备。

这些人并不关心Claude写了多少代码。**代码只是Agent为完成任务临时使用的工具。**

他们真正需要的，是把目标和资料交给Claude，让它自己查找信息、编写脚本、调用工具，最后交付结果。甚至一些没有编程经验的用户，也开始主动学习这个有门槛的编程工具。

到2025年末，Anthropic内部已有多个团队探索如何让Claude从回答问题走向执行任务，也积累了桌面应用、虚拟机、规划工具、Skills和一批未公开的原型。

真实的用户信号让团队意识到，通用知识工作Agent的需求已经出现，发布时机也逐渐成熟。

Boris提出，可以把这些组件与Claude Code组合起来，做一个非程序员也能使用的版本。



他们希望尽快做出这款产品的原型。

负责Claude桌面产品的Felix Rieseberg和一个小团队开始赶工。他们没有重新开发一套Agent，而是**直接复用Claude Code的agent harness**，再把此前散落在不同团队里的组件组装起来。

每名工程师会同时运行3—8个Claude Code Agent：一个开发界面，一个处理后端逻辑，一个调研技术方案，其他Agent则修复内部用户刚刚在Slack里报告的问题。工程师主要负责划分任务、作出产品决策和验收结果。

10天后，Cowork完成开发。但它并非在10天里凭空诞生：此前一年多积累的模型能力、Claude Code的Agent能力、各团队做过的原型和基础设施，都已经就位。**团队只是等到能力与需求同时越过阈值，把它们收拢成了一款产品。**

Cowork也以研究预览版发布。它把Claude带进了比代码库更模糊的工作环境：

模型不仅要调用工具，还要理解不够明确的要求，判断何时询问用户，并交付真正可用的文档、表格和演示文稿。

由此形成的真实用例、暴露的失败，也会反馈给下一轮模型、工具和agent harness的改进。

## 6、Anthropic 如何规划下一代模型？

在传统软件公司里，路线图通常长得很具体——下个季度上线什么功能、哪个版本改哪个界面、哪些需求排进下一次发布。

Dianne Penn表示，**Anthropic并不会从具体的版本和功能开始规划。**

团队会先把版本号暂时放到一边，设想一个更远的场景：如果Claude最终真的能够承担某项工作，它应该会做什么？这张想象中的规划图，则会由一条条能力轴组成。

Dianne 用整理日历来说明这种区别。

要替用户重排工作日历，Claude 必须先读懂页面和日程，理解会议内容，发现时间冲突，判断哪些会议可以移动。如果过程中出现新的冲突，还要及时调整原计划。

因此，“重排日历”并不是一项孤立功能，而是**视觉识别、语言理解、工具调用、错误恢复等能力共同作用的结果。**

内部讨论的重点，不是“把日历功能放进哪个版本”，而是判断这些基础能力的成熟度。

如果当前模型已经接近目标，剩余缺口或许可以通过增加数据、强化学习和后训练补齐；如果模型只能完成其中很小一部分，问题可能仍在基础研究或训练方法层面。

Anthropic 的产品、工程和研究团队会共同判断能力缺口，再决定下一步是补充数据、开展强化学习，还是回到更底层的研究。

Alex Albert用了一个很不软件工程的词来形容Anthropic打造模型的过程——**培养**。Alex 也是Anthropic的一名技术研究产品经理。他提到，即使团队已经决定了训练设置、技术路线和模型架构，也只能对结果形成直觉。



模型进入训练后，才会逐渐显露出擅长与不擅长的领域，有时还会出现未被预料的行为。它不是按照图纸组装出来的功能，更像一个在训练环境、反馈信号和约束条件下成长的复杂系统。

因此，Research PM 必须从模型构思阶段就进入流程。

Alex 不能等模型训练完成后再寻找市场，而要在训练开始前参与定义“模型需求”。

与此同时，每一代模型还要解决上一代暴露的问题。团队既要思考“希望模型具备什么能力”，也要不断确认“它实际上长成了什么样”。

Research PM 的视野还必须跨越所有产品界面。同一个底层模型，进入 Claude.ai、API、Claude Code 和 Cowork 后，会获得不同的系统提示、工具、上下文和工作方式，最终表现也可能截然不同。

Anthropic 打造的并不是一颗孤立的模型，而是“**模型与运行环境**”组成的完整系统。

## 7、Co-design Loop：如何让模型与产品共同演进？

支撑模型与产品协同创新的，不只是一组特殊岗位和 workflows，还有 Anthropic 的文化理念。

AI 大幅降低了原型成本。一个想法不必先争取完整团队和大笔资源，而是可以先做出来，交给同事使用，再根据真实行为判断是否值得继续。

Anthropic 内部极为重视这种「原型文化」。除了产品和工程团队，研究、销售、招聘等部门的员工，也会主动搭建各种工具原型。

Alex 用“**让千朵花绽放**”形容这种文化。原型也是观察模型能力的一种方法。

模型在研究实验中获得新能力，被内部员工迅速放进新的工具、流程和场景。那些真正有用的原型会吸引更多人使用，暴露出实验室里难以预料的用法和失败，并为下一轮产品与模型改进提供信号。

Claude Code 从个人工具走向内部流行，再成为正式产品，就是这种文化最典型的结果。

产品进入真实世界后，另一个问题随之出现：**反馈规模远远超过人工处理能力**。Anthropic 显然很重视用户反馈。

数以百万计的用户通过不同渠道使用 Claude。如果只靠产品经理逐条阅读，反复出现的问题很容易被情绪表达、偶发故障和语境差异淹没。

Alex 提到，如今他们更多地使用 Claude 分析关于 Claude 的反馈。

团队先让模型对大量反馈进行分组和聚类，找出反复出现的主题，再把零散问题抽象成可以诊断的失败类型。

有时，几十个案例已经足以证明某种缺陷稳定存在。它们未必能精确代表线上总体失败率，却能把“感觉它不太行”变成“**这个行为可以被复现和验证**”。

这形成了一个看似奇特的循环：Claude 帮助产品经理分析 Claude，生成测试 Claude 的问题，甚至辅助评估 Claude 的回答。



但模型不会因此自动决定产品路线。聚类是否合理、失败是否真正影响用户、样本是否贴近真实任务，以及问题是否值得占用昂贵的训练资源，仍然需要产品经理和研究人员作出判断。

原型越多，模型能力被发现得越早；真实使用越深入，反馈越具体；eval越贴近任务，下一轮训练越有方向。

**模型与产品由此不再彼此等待，而是在同一个co-design loop中共同演进。**

如今，这套循环，正在更多 AI 公司内部转动起来。

比如，姚顺雨加入腾讯后，着力推动模型与产品 Co-design，让模型研发更贴近真实需求。

这一思路已在 Hy3 和 WorkBuddy 上初见成效：Hy3在WorkBuddy 首发上线后，自选模型用户中选择Hy3的占比达到60%。

调用量增长带来更多真实反馈，推动 Hy3 持续迭代。更强的模型能力，又进一步改善了 WorkBuddy 的体验。

类似的转变也发生在马斯克的 AI 版图中。核心团队流失、产品落地不畅，曾让 xAI 陷入低谷，马斯克也坦言要“从地基开始重建”。

Cursor 的加入成为转折点。它补上了真实任务入口，也让 SpaceX AI 跑通开发者反馈、模型训练与产品验证的闭环。

AI 产业最早比的是理论突破与技术路线，随后比的是算力规模与数据质量。

下一阶段，胜负或许更多取决于——**产品与模型的齿轮能否真正咬合，循环能否转得足够快，以及每一次循环，能否成为下一次加速的复利。**

本文来源：[深流研究所](#)

风险提示及免责声明

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限免

 73    收藏

分享到:  

写评论

请发表您的评论

 表情    图片

发布评论